Search Engine Project

Big Data | 3ºGCID | ULPGC | 2026/2027 | Stage 2: Building a Service-Oriented Architecture

—

Group name: TheSeekerOfDarkness

1. Dennis Mertens · L73M12CY
2. Pablo Mellado Martin · 79087666G
3. Hugo Barreto Suárez · 51167208M

Github



2. Introduction and Objectives

The primary objective of Stage 2 is to transform the foundational data layer designed in Stage 1 into a fully operational, decoupled, and robust **Service-Oriented Architecture (SOA)**. While Stage 1 focused on benchmark comparisons between programming languages (Python, Java, Rust) and physical storage layouts (monolithic, hierarchical, database), Stage 2 modularizes the search engine pipeline into autonomous, cooperating microservices implemented in Java 17.

The system transitions from an in-process script execution model to an ecosystem of distributed network services communicating through HTTP/REST interfaces with structured JSON contracts. All processes run on a single host machine, establishing the necessary concurrency, transaction isolation, and synchronization patterns required before moving to a distributed cluster in Stage 3.

The core milestones accomplished in this stage include:

- **Pipeline Decomposition**: Separating responsibilities into three independent microservices: Ingestion Service, Indexing Service, and Search Service, each packaged as an autonomous executable fat-JAR using the minimalist Javalin 6.1.3 web framework.
- **Explicit Coordination**: Designing and implementing a hybrid orchestration strategy driven by time-based background synchronization (via scheduled daemon polling) combined with idempotent internal REST commands.
- **Front-End API Contract**: Exposing a comprehensive public search interface featuring boolean term conjunction (AND), multi-field metadata filtering (author, language, release year), and deterministic result pagination.
- **Storage Integrity and Concurrency**: Consolidating bibliographic metadata and the inverted index into an optimized SQLite relational datamart operating in Write-Ahead Logging (WAL) mode to guarantee transactional atomicity, non-blocking concurrent reads, and crash resilience.
- **Empirical Microbenchmarking**: Rigorously benchmarking algorithmic throughput and latency using the Java Microbenchmark Harness (JMH) to characterize component boundaries under controlled JVM executions.

3. System Architecture

The architecture enforces a strict separation of concerns, ensuring that data ingestion, natural language processing/indexing, and query resolution operate as independent processes with explicit interface boundaries and lifecycle management.

3.1. Microservice Responsabilities

- **Ingestion Service (Port 7001):** Dedicated to fetching raw text from Project Gutenberg mirrors. It cleans legal disclaimers, partitions text into standardized header and body files, and records them in the time-based hierarchical datalake (datalake/YYYYMMDD/HH/).
- **Indexing Service (Port 7002):** The system's computational core. It scans the datalake, parses bibliographic headers into structured records, tokenizes text into normalized stems, eliminates 127 stop words, and safely populates the SQLite datamart within isolated transactions.
- **Search Service (Port 7003):** The user-facing gateway. It receives search queries, tokenizes terms using the exact same stop-word dictionary, and executes compound SQL joins against the relational datamart to return paginated, filtered bibliographic matches.

Here is the detailed architectural map of the single-host distributed system, illustrating the API boundaries, execution environments, and transactional storage interfaces:

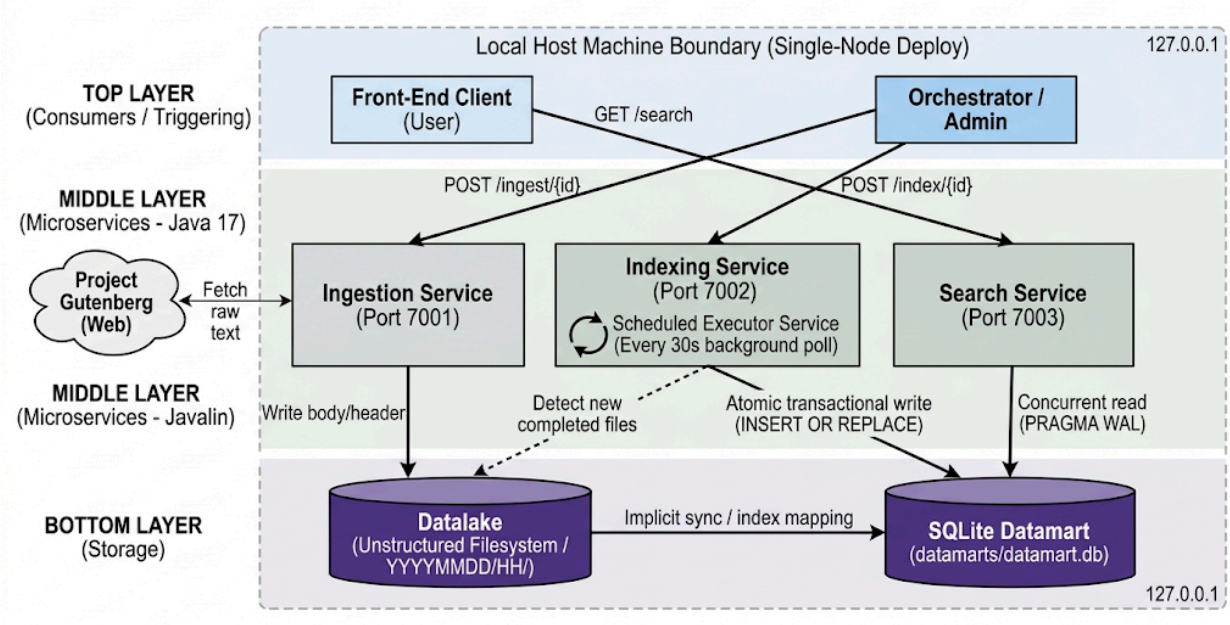


Figure 0: Hybrid Time-Based Service-Oriented Architecture (SOA) with Admin Overrides on a Single-Node Host.

3.2. Coordination Strategy: Hybrid Time-Based Orchestration

The system implements a **Time-based Coordination** mechanism with internal REST actuation. Rather than introducing a heavy external broker (such as Kafka or RabbitMQ) on a single-node host, the Indexing Service incorporates an internal ScheduledExecutorService that wakes up every 30 seconds (configurable via SYNC_INTERVAL_SECONDS).

During each cycle, the scheduler inspects the datalake directory structure, identifies downloaded book IDs, and evaluates completion markers (ensuring both .header.txt and .body.txt are closed and non-empty). It cross-references these with the indexed book IDs in SQLite and automatically triggers batch indexing for all pending items. Additionally, internal endpoints (POST /index/{book_id} and POST /index/all) allow administrative or external controllers to force synchronous updates without waiting for the next polling window.

4. API Design and Contracts

The complete contract is formalized according to the OpenAPI 3.0.3 specification in the openapi.yaml file at the repository root. Interfaces are split into front-end-facing and internal service categories.

4.1. Front-End Facing API (Search Service)

Method & Route	Parameters	Payload / Response Structure	Status Codes

GET /search	q (string, required)
	author (string, optional)
	language (string, optional)
	year (int, optional)
	limit (int, default: all)
	offset (int, default: 0)
		{
		"query": "adventure",
		"filters": {"language": "en"},
		"total_matches": 14,
		"offset": 0, "limit": 10,
		"results": [{ "book_id": 11, ... }]
		}
			200 OK: Valid search (returns empty list if 0 matches).
			400 Bad Request: Missing q or non-numeric year.
			500 Error: Datamart connection error.

4.2. Internal Service and Operational APIs

Service	Endpoint	Description & Workflow Role	Codes & Semantics
Ingestion (7001)	`POST /ingest/{id}`	Downloads book from Project Gutenberg, splits text into header/body, and writes to datalake.	**200:** Downloaded / Exists. **422:** Missing markers. **502:** Network failure.
Ingestion (7001)	`GET /ingest/status/{id}`	Verifies whether a book ID exists completely in the datalake.	**200:** Available. **404:** Not found.
Ingestion (7001)	`GET /ingest/list`	Lists all downloaded book IDs present across all datalake folders.	**200:** `{"count": N, "books": [...]}`
Indexing (7002)	`POST /index/{id}`	Synchronously indexes a single book ID from the datalake into the datamart.	**200:** Indexed. **404:** Missing from datalake. **500:** Transaction failure.
Indexing (7002)	`POST /index/all`	Triggers full batch indexing across all unindexed datalake books.	**200:** Summary with processed/failed IDs.
Indexing (7002)	`GET /index/status/{id}`	Checks whether a specific book ID has been indexed in SQLite.	**200:** Indexed. **404:** Not indexed.
Indexing (7002)	`GET /index/book/{id}`	Retrieves extracted metadata for a specific book ID.	**200:** Bibliographic metadata object. **404:** Not indexed.
All (7001-7003)	`GET /status`	Operational health-check endpoint for service monitoring and scripts.	**200:** `{"service": name, "status": "running"}`

4.3. Interface Validation Evidence

The API interfaces were exercised and validated using automated integration checks and HTTP CLI interactions. The test suite demonstrates compliance across all edge cases:

- **Single and Multi-term Conjunction**: GET /search?q=pride+prejudice strictly returns books containing both terms.
- **Filtered Queries**: GET /search?q=novel&author=Austen&language=en&year=1813 returns only book 1342.
- **Zero Matches Handling**: Queries with disjoint terms (e.g., q=darcy+frankenstein) return 200 OK with "total_matches": 0 and "results": [].
- **Malformed Parameters**: Omitting the query string (GET /search) yields 400 Bad Request with a descriptive JSON error message.
- **Pagination**: GET /search?q=classic&limit=2&offset=2 returns the requested window without offset drift.

4.4. Non-HTTP Coordination Contract (Time-Based Background Polling)

The scheduler in `Indexing Service` enforces a strict implicit synchronization contract with the shared local storage:

- **Trigger Mechanism**: Non-HTTP daemon execution through a Java ScheduledExecutorService running a single-thread poll every 30 seconds (SYNC_INTERVAL_SECONDS).
- **Completeness Contract (Datalake Boundary):** A book with ID book_id is only deemed "indexed-ready" when the crawler completes writing without interruption and both <book_id>.header.txt and <book_id>.body.txt exist in datalake/YYYYMMDD/HH/ and are larger than 0 bytes.
- **Coordination Delta Flow**: The service evaluates downloaded (by scanning directories) vs. indexed (the internal list query against books in SQLite). Sub-differences (new data) trigger a synchronous update(bookId) batch loop.
- **Recovery and Safety**: Incomplete writes (crashed midway) result in missing or zero-byte files, making the book invisible to the polling worker until re-ingested. Repeated scanning will simply attempt to overwrite in a non-disruptive, idempotent operation.

5. Microservice Implementation

Each microservice is structured as an autonomous Maven module producing an executable jar via maven-shade-plugin. The shared technological stack relies on:

- **Java 17 (LTS):** Core language utilizing modern records, pattern matching, and concurrent utilities.
- **Javalin 6.1.3:** Lightweight, unopinionated web framework wrapping an embedded Jetty HTTP engine with near-zero routing overhead.
- **Google Gson 2.11.0**: High-performance JSON serialization and deserialization.
- **Xerial SQLite JDBC 3.46.0**: Native SQLite driver supporting embedded relational transactions

5.1. Task Ownership, Idempotency, and Crash Recovery

To prevent data corruption during concurrent execution or abrupt service crashes, all modifications to the datamart are executed within strictly isolated transactions:

```
try (Connection connection = DriverManager.getConnection(url)) {
    connection.setAutoCommit(false);
    try {
        metadata.save(connection, book);    // Executes: INSERT OR REPLACE INTO books ...
        index.add(connection, bookId, body); // Executes: DELETE FROM inverted_index
WHERE book_id = ?
                                    // Followed by: INSERT INTO inverted_index (term, book_id) ...
        connection.commit();
    } catch (SQLException e) {
        connection.rollback();
        throw e;
    }
}
```

This transactional pattern guarantees **idempotency**: reindexing an existing book replaces its metadata catalog entry and purges prior inverted index postings before writing the new term batch. If the indexing process fails midway, the database rolls back cleanly, leaving no orphaned postings.

5.2. Consistent Update Publication

The SQLite datamart is configured with PRAGMA journal_mode=WAL; and a 60-second busy timeout (PRAGMA busy_timeout=60000;). In Write-Ahead Logging mode, reads from the Search Service never block writes from the Indexing Service, and writes never block readers. Uncommitted write transactions remain in the separate WAL journal file, making newly indexed books atomically visible to search queries precisely when connection.commit() succeeds.

5.3. Fault Tolerance, Network Retries, and Error Resilience

Given that Project Gutenberg is an external remote resource, the system incorporates automated fault-tolerance mechanisms to ensure pipeline resilience in Ingestion Service's GutenbergDownloader:

- **Connection & Read Timeouts**: All outbound HTTP fetch operations are configured with a strict 10-second connect timeout and a 30-second read timeout to prevent thread exhaustion if the Gutenberg mirror goes offline or runs slow.
- **Retry Exponential Backoff**: Transient errors (HTTP 5xx, gateway timeouts, socket resets) trigger up to 3 automatic download retries, waiting 2 attempt seconds between each.
- **Error Categorization**: Network failures or non-200 responses that fail after all retry attempts return a JSON with status 502 Bad Gateway. If a requested book simply does not exist on Gutenberg, the service cleanly aborts with a 404 Not Found response.
- **Sanitization Fallbacks**: Book headers or bodies with corrupted text or non-UTF-8 encoding are sanitized with fallback "Unknown" markers during the extraction step, ensuring that bad source files never trigger a pipeline crash.

6. Design Decisions

6.1. Justification of the Coordination Mechanism

A hybrid approach was chosen over complex message brokers (e.g., ActiveMQ, Kafka). On a single physical machine, message brokers introduce JVM memory overhead, additional port management, and external point-of-failure risks without providing performance benefits. By utilizing a daemon ScheduledExecutorService inside the Indexing Service paired with filesystem readiness checks, services remain completely decoupled while operating autonomously. Internal REST endpoints supplement the scheduler for deterministic integration testing.

6.2. Storage Consolidation: Single SQLite Datamart

In Stage 1, monolithic JSON, hierarchical folders, and MongoDB were compared. In Stage 2, both metadata and inverted index terms are unified into a single SQLite database (datamarts/datamart.db). This consolidation allows complex searches—combining inverted index term intersection with metadata attribute filtering—to execute as a single, highly optimized SQL statement:

```
SELECT b.book_id, b.title, b.author, b.language, b.year, b.datalake_path
FROM books b
JOIN (
    SELECT book_id FROM inverted_index
    WHERE term IN (?, ?, ...)
    GROUP BY book_id
    HAVING COUNT(DISTINCT term) = ?
) m ON b.book_id = m.book_id
WHERE b.author LIKE ? AND b.language = ? AND b.year = ?
ORDER BY b.book_id ASC LIMIT ? OFFSET ?;
```

B-tree indexes on inverted_index(term, book_id) and inverted_index(book_id) allow SQLite to resolve term conjunctions in memory without loading multi-megabyte JSON structures into Java heap space.

6.3. Concurrency Model and Contention Analysis

Each Javalin service instance utilizes an internal Jetty thread pool to handle concurrent HTTP requests.

- **Search Concurrency**: Highly scalable. Under SQLite WAL mode, multiple worker threads in the Search Service can perform concurrent read operations without locking table locks.
- **Indexing Contention**: Inherently serialized. SQLite enforces a single-writer lock per database file. While multi-threaded tokenization in memory provides speedups, simultaneous database writes must be synchronized to prevent SQLITE_BUSY exceptions.

7. Benchmarks and Results

7.1. Experimental Environment and Host Specification

All microbenchmarks and service-level integration experiments were executed on a dedicated host machine. All microservices, SQLite datamarts, and the test workload generator (an automated Python benchmarking client) were executed concurrently on the same system.

Parameter	Specification & Configuration
Host Processor (CPU)	AMD Ryzen 5 5625U (6 physical cores, 12 logical threads @ 2.3 GHz base, up to 4.3 GHz boost)
Installed Memory (RAM)	15.3 GB DDR4
Storage Subsystem	512 GB NVMe PCIe M.2 SSD
Operating System	Microsoft Windows 11 Home (64-bit)
Java Runtime Environment	OpenJDK 64-Bit Server VM (build 21.0.10+7-LTS, mixed mode)
Target Dataset	107 Project Gutenberg books (unstructured text in `datalake/`)
Consolidated Datamart	`datamarts/datamart.db` (39.9 MB, 769,838 posting rows, 195,879 distinct terms)

CPU Measurement Unit	Expressed as % of one logical core (e.g., 241% equals ~2.4 saturated logical cores)
Query Workload Mix	2,000 HTTP requests following 200 warmup runs across 10 query archetypes (single term, conjunctions, metadata filters, pagination, zero matches)

7.2. Component Microbenchmarks (JMH 1.37)

To measure pure algorithmic performance without network protocol overhead, microbenchmarks were executed using JMH 1.37 configured with 5 warmup iterations, 10 measurement iterations, and 1 fork:

Benchmark Class	Target Operation	Mode	Execution 1 (Mean ± Error)	Execution 2 (Mean ± Error)	Unit
`IndexerBenchmark`	`benchmarkIndexSingleBook` (ID: 1342)	Throughput	10.56 ± 0.55	11.38 ± 1.41	ops/s (books/s)
`SearchBenchmark`	`benchmarkSearchQuery` (1,000 indexed books)	AverageTime	4.83 ± 0.16	4.88 ± 0.15	ms/op

The JMH microbenchmarks confirm that in-process SQLite queries resolve in under 4.9 ms for synthetic corpora. Single-book parsing and inverted index generation achieve ~11 books/second in pure JVM computation when isolated from HTTP transfer overhead.

7.3. Indexing Service Throughput and Idempotency Validation



Figure 1: Indexed books per second via HTTP requests to the Indexing Service (1 client: 2.18 books/s, 4 clients: 2.63 books/s, Re-index 1 client: 1.11 books/s).

Workload Test	Elapsed Time	Throughput	Mean Latency	p95 Latency	Process CPU	Process RAM (Mean/Max)	Host CPU
1 Client	49.1 s	2.18 books/s	459 ms	883 ms	90%	259 / 383 MB	35%
4 Concurrent Clients	40.7 s	2.63 books/s	1,234 ms	3,479 ms	120%	270 / 348 MB	45%
Re-indexing (1 Client)	96.5 s	1.11 books/s	—	—	—	—	—

Write Lock Contention: Increasing concurrency from 1 to 4 clients resulted in a marginal throughput gain of only +20.6% (from 2.18 to 2.63 books/s) while average latency degraded by 2.7x and 95th percentile latency exploded from 883 ms to 3,479 ms. This confirms that SQLite's single-writer architecture serializes write transactions, turning the local disk database into the primary system bottleneck.

Idempotency Verification: Executing a complete re-indexing pass across already indexed books took 96.5 s and resulted in exactly **769,838 rows** in the inverted_index table. The row count was identical before and after the re-index, verifying that the transactional INSERT OR REPLACE and pre-delete mechanisms guarantee 100% idempotency without duplicate postings.

7.4. Search Concurrency, Latency Distributions, and Replication

The Search Service was evaluated under 1, 4, 8, and 16 concurrent clients executing 2,000 queries over the 107-book corpus. A multi-instance replication test was also performed by running two Search Service processes concurrently on ports 7003 and 7004.

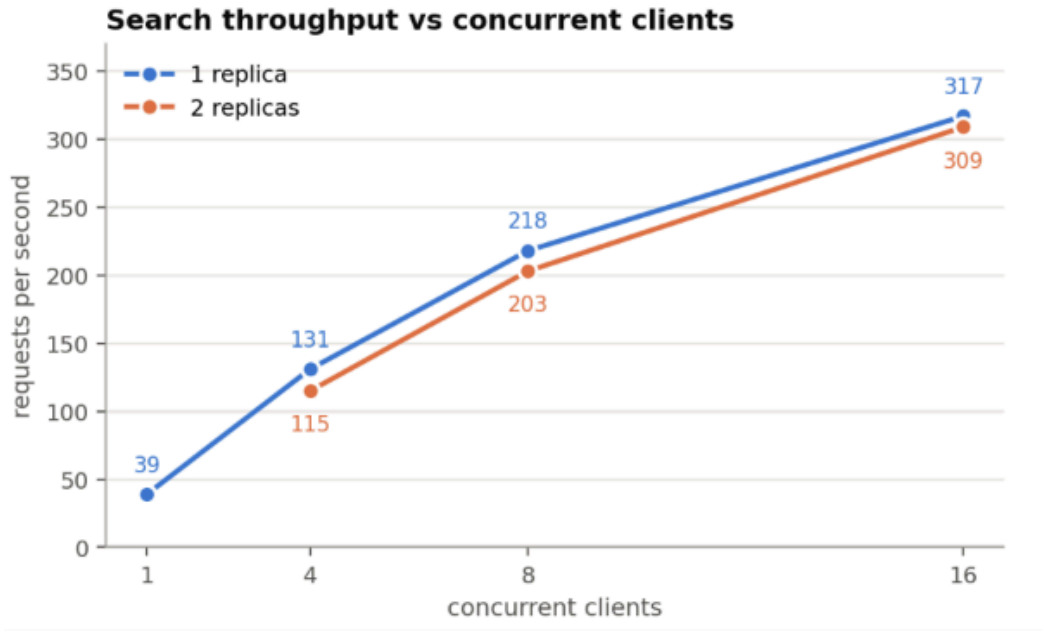


Figure 2: Search requests per second across concurrent clients (1, 4, 8, 16) comparing 1 replica vs. 2 replicas.

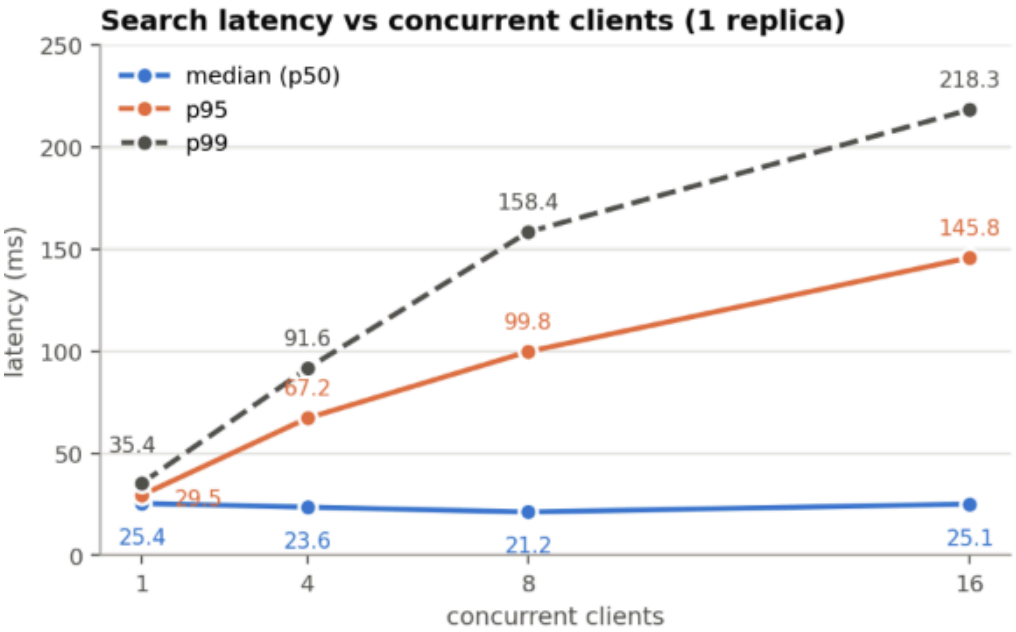


Figure 3: Search query latency distribution (median p50, p95, and p99 in ms) across concurrent clients for replica.

Deployment	Clients	Throughput	Median (p50)	p95	p99	Process CPU	RAM (Mean)	Host CPU
1 Replica	1	39 req/s	25.4 ms	29.5 ms	35.4 ms	57%	179 MB	42%
1 Replica	4	131 req/s	23.6 ms	67.2 ms	91.6 ms	126%	229 MB	46%
1 Replica	8	218 req/s	21.2 ms	99.8 ms	158.4 ms	181%	238 MB	65%
1 Replica	16	317 req/s	25.1 ms	145.8 ms	218.3 ms	241%	238 MB	84%
2 Replicas	4	115 req/s	26.0 ms	72.7 ms	102.8 ms	65% / 88%	234 / 132 MB	55%
2 Replicas	8	203 req/s	22.4 ms	102.3 ms	141.9 ms	84% / 104%	233 / 197 MB	67%
2 Replicas	16	309 req/s	26.3 ms	152.5 ms	269.7 ms	114% / 132%	237 / 218 MB	71%

Reliability and Query Responsiveness: Across all runs, **zero HTTP errors** occurred (100% success rate). The median latency (p50) remained exceptionally stable between 21.2 ms and 26.3 ms regardless of concurrency level or replica count, demonstrating that SQLite WAL read transactions handle concurrent B-tree lookups without locking degradation. Tail latencies (p95 and p99) grew proportionally under load due to OS thread scheduling and HTTP connection queueing.

Single-Host Replication Ceiling: Deploying two Search Service replicas did not improve throughput (yielding 309 req/s vs. 317 req/s at 16 clients). Because a single Javalin instance already distributes incoming requests across multiple worker threads (consuming up to 241% CPU, or ~2.4 cores), adding a second JVM instance increased memory overhead and context switching on a physical host operating at 84% total CPU saturation (including the benchmarking client).

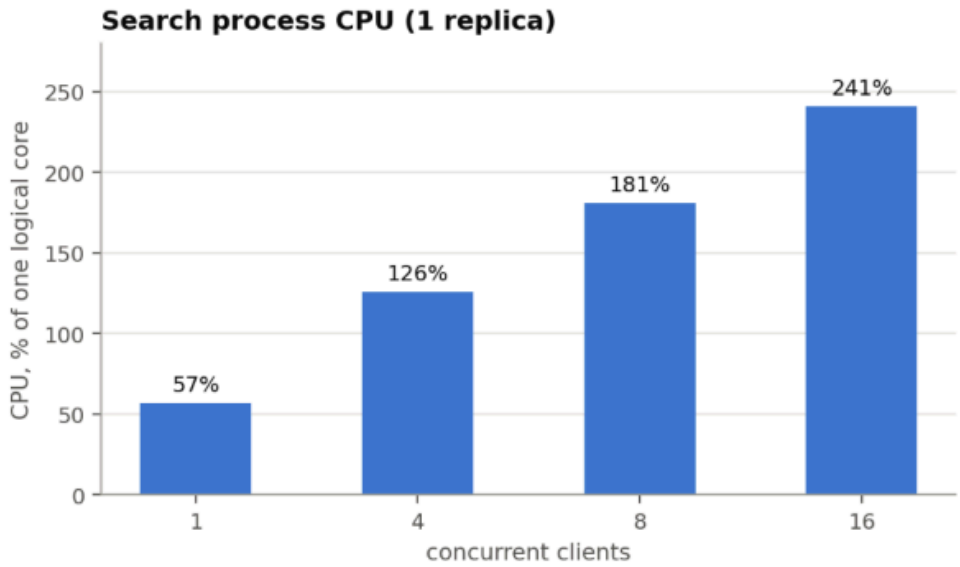


Figure 4: Search process CPU utilization (% of one logical core) under 1, 4, 8, and 16 concurrent clients (57%, 126%, 181%, and 241%).

7.5. Coordination Overhead and Pipeline Latency

The time elapsed from when the Ingestion Service finished writing a book to the datalake until it became searchable in SQLite was tracked under the 30-second background polling cycle:

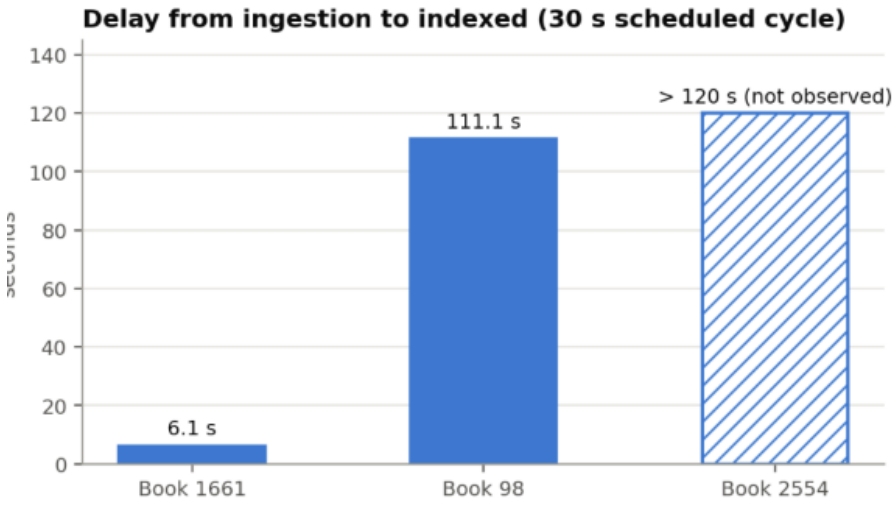


Figure 5: Time elapsed (seconds) from book ingestion completion until indexed in the datamart (Book 1661: 6.1 s, Book 98: 111.1 s, Book 2554: > 120 s).

- **Book 1661**: 6.1 seconds delay (ingestion coincided with the start of a scheduled cycle).
- **Book 98:** 111.1 seconds delay.
- **Book 2554**: > 120.0 seconds delay (exceeded the observation window).

Root Cause Analysis: In the current implementation, the background coordinator executes updateAll(), which re-evaluates all books in the datalake rather than processing only unindexed delta files. With 107 books, a full scan cycle requires ~96.5 s plus the 30 s idle interval. This empirical finding demonstrates that delta-based tracking is essential to keep ingestion-to-index latency low.

8. Conclusions and Future Improvements

8.1. Measured Architecture Evaluation

The Stage 2 service-oriented architecture achieved its core functional objectives:

- **High Search Scalability**: The Search Service achieved up to 317 req/s with a sub-26 ms median response time and zero errors under 16 concurrent clients, validating SQLite WAL mode for read-heavy workloads.
- **Transactional Safety & Idempotency**: Re-indexing the full 107-book corpus maintained exactly 769,838 index postings without generating duplicates or corrupting data.
- **Low Resource Footprint**: Microservices maintained modest memory profiles (~238 MB for Search, ~270 MB for Indexing).

However, empirical profiling highlighted critical bottlenecks inherent to a single-machine deployment:

- **Single-Writer Database Lock**: Indexing throughput plateaued at 2.63 books/s due to SQLite write serialization. Adding concurrent indexing workers increased latency rather than throughput.
- **Host Resource Saturation**: Running multiple search replicas on one machine offered no throughput benefit because a single multithreaded Javalin process was already sufficient to saturate the available CPU cores.
- **Full-Scan Scheduling Overhead**: The background polling cycle introduced indexing delays exceeding 111 seconds due to re-scanning existing datalake files.
- **Metadata & Interface Limitations**: Extracted book publication years reflect Project Gutenberg release dates rather than historical publication dates, and the search endpoint returns host-specific local datalake paths.

8.2. Evolution for the Stage 3 Distributed Cluster

To transition this architecture to an elastic multi-node cluster in Stage 3, the following design adaptations are required:

1. **Distributed Storage Abstraction**: Migrate the local filesystem datalake (datalake/) to a distributed object store (MinIO/S3 or HDFS) so that any cluster node can access raw text independent of physical locality.
2. **Partitioned / Multi-Writer Datamart**: Replace the single SQLite database with a distributed datamart (e.g., MongoDB with sharding or PostgreSQL with partitioned tables and read replicas) to eliminate the single-writer lock.
3. **Event-Driven Broker Choreography**: Replace the time-based polling scheduler with a message broker (Apache Kafka or RabbitMQ). The Ingestion Service will emit BookDownloaded events containing book IDs, allowing multiple distributed Indexing workers to process incoming books in parallel without redundant full scans.
4. **Cluster Load Balancing**: Position a reverse proxy or API Gateway (Nginx or Envoy) in front of horizontally replicated Search Service instances across distinct physical nodes.